

1 차시	OpenCV 정규화 및 Object Detection 입문	총 2 문제	연습: ☐	과제:	평가: ☐
------	--	--------	------------------------------	-----	------------------------------

* 프로그램 작성 문제는 코드와 실행 결과 화면을 캡처해 답란에 붙여넣어주세요.

문제 1) 아날로그 신호를 디지털 영상으로 변환하는 과정을 올바른 순서로 나열한 것은?

- ① Encoding → Sampling → Quantization
- ② Quantization → Sampling → Encoding
- ③ Sampling → Quantization → Encoding
- ④ Sampling → Encoding → Quantization
- ⑤ Encoding → Quantization → Sampling

>> 3

문제 2) 히스토그램 정규화/평활화(histogram normalization / equalization)를 수행하는 주된 목적 무엇인지 한 문장으로 설명하시오.

>> 데이터가 균형 있게 분포하도록 전처리하기 위해서이다.

2 차시	OpenCV 기하학적 변환	총 2 문제	연습: ☐	과제:	평가: ☐
------	-------------------	--------	------------------------------	-----	------------------------------

* 프로그램 작성 문제는 코드와 실행 결과 화면을 캡처해 답란에 붙여넣어주세요.

문제 1) 다음 중 Smoothing Filter 에 대한 설명으로 옳지 않은 것은?

- ① 평균(Mean) 필터는 커널 내 픽셀들의 평균값으로 중앙 픽셀을 대체하여 노이즈를 줄인다.
- ② 가우시안(Gaussian) 필터는 가우시안 분포를 가중치로 사용하여, 중심부에 더 큰 가중치를 부여한다.
- ③ 중간값(Median) 필터는 커널 내 픽셀들의 중앙값으로 대체하여, 특히 Salt-and-Pepper 노이즈 제거에 효과적이다.
- ④ 양방향(Bilateral) 필터는 공간적 거리뿐 아니라 픽셀 값 차이도 고려하여, 엣지를 유지하면서 블러링한다.
- ⑤ 블러 필터는 엣지를 강화(sharpening)하기 위한 필터로, 영상의 세부 디테일을 강조하는 것이 목적이다.

>> 5

문제 2) 아래의 물음들에 답하시오.

2-1. 다음 중 영상의 회전(Rotation)과 확대·축소(Scaling)에 대한 설명으로 옳은 것은?

- ① 회전은 항상 영상의 왼쪽 위 모서리를 기준으로 수행되며, 중심을 기준으로 회전할 수는 없다.
- ② cv2.getRotationMatrix2D 함수는 회전에 사용되는 3×3 동차 좌표계 행렬을 반환한다.
- ③ 확대·축소 시에는 새로운 좌표에 대응하는 픽셀 값을 채우기 위해 보간(interpolation)이 필요하며, 최근접/선형/큐빅 보간 등이 사용될 수 있다.
- ④ 스케일링은 픽셀 값을 직접 곱하거나 나누는 연산이므로, 별도의 보간 과정이 필요 없다.
- ⑤ 회전은 기하학적 변환이 아니라 필터링의 일종이다.

>> 3

2-2. 영상의 확대·축소(scaling) 에서 “보간법(interpolation)”이 필요한 이유를 간단히 설명하고,

대표적인 보간법 2 가지만 쓰시오.

>> 영상을 확대하거나 축소하면 기존 픽셀과는 완전히 변형되는 픽셀들이 생기기 때문에 보간법으로 이 픽셀들을 해결한다. 대표적인 보간법으로는 Nearest 보간법과 Linear 보간법, Cubic 보간법 등이 있다.

3 차시	OpenCV 영상 처리, Object Tracking	총 2 문제	연습: ☐	과제:	평가: ☐
------	----------------------------------	--------	------------------------------	-----	------------------------------

*** 프로그램 작성 문제는 코드와 실행 결과 화면을 캡처해 답란에 붙여넣어주세요.**

문제 1) 다음 설명을 읽고 빈칸(A)을 채우세요.

캐니(Canny) 엣지 검출은 노이즈 제거, 그래디언트 계산, (A), 이중 임계값 처리 순으로 진행되며, (A) 단계는 엣지의 방향을 따라 가장 강한 픽셀만 남기는 단계이다.

>> 비최대 억제(NMS)

문제 2) 엣지(edge) 검출 시 단순 차분 연산이 아닌 소벨(Sobel) 필터를 사용하는 이유를 간단히 설명하세요.

>> 소벨 필터는 1 차 미분으로 gradient 를 계산하여 엣지 검출에 있어 더 좋은 성능을 보이기 때문이다.

4 차시	추적 알고리즘 실습	총 2 문제	연습: ☐	과제:	평가: ☐
------	------------	--------	------------------------------	-----	------------------------------

* 프로그램 작성 문제는 코드와 실행 결과 화면을 캡처해 답란에 붙여넣어주세요.

문제 1) 다음 중 CamShift 알고리즘의 특징으로 옳은 것을 모두 고르세요.

1. 윈도우 크기가 고정되어 있다.
2. 객체 회전에 대응할 수 있다.
3. Hue 기반 히스토그램을 주로 사용한다.
4. MeanShift 보다 계산량이 적다.
5. 프레임마다 윈도우 크기·각도를 업데이트한다.

>> 2, 3, 5

문제 2) Lucas-Kanade Optical Flow 수행 전 `cv2.goodFeaturesToTrack()` 을 사용하는 이유를 간단히 설명하세요.

>> `cv2.goodFeautresToTrack()` 으로 영상의 좋은 특징점들을 자동으로 추출시켜 LK optical flow 를 수행하면 그 특징점들을 기반으로 객체 움직임을 안정적으로 추적 가능하기 때문이다.

5 차시	CNN 기반 분류 앱	총 2 문제	연습: ☐	과제:	평가: ☐
------	-------------	--------	------------------------------	-----	------------------------------

* 프로그램 작성 문제는 코드와 실행 결과 화면을 캡처해 답란에 붙여넣어주세요.

문제 1) 다음 두 모델의 같은점, 차이점을 하나씩 작성하세요.

- VGGNet
- GoogLeNet (Inception v1)

>> (1) 공통점: 이미지 분류를 위한 CNN 모델이다.

(2) 차이점: VGGNet 은 3x3 Conv 를 반복하는 단순하고 깊은 구조이고, GoogLeNet 은 inception 모듈을 사용해 다양한 커널을 병렬로 연결했기 때문에 VGGNet 보다 훨씬 적은 수의 파라미터를 사용한다.

문제 2) 아래 설명이 의미하는 모델을 고르고, 해당 모델의 단점도 간단히 작성하세요.

“병렬 합성곱 구조를 활용하며 1×1 합성곱으로 채널 연산량을 줄이고, 전역 평균 풀링을 사용해 매개변수를 크게 감소시킨다.”

1. LeNet
2. AlexNet
3. VGGNet
4. GoogLeNet(Inception v1)

>> 4. 단점: 구조가 복잡해서 수정이 어렵다.

6 차시	신경망 스타일 전이	총 2 문제	연습: ☐	과제:	평가: ☐
------	------------	--------	------------------------------	-----	------------------------------

* 프로그램 작성 문제는 코드와 실행 결과 화면을 캡처해 답란에 붙여넣어주세요.

문제 1) Style Loss 계산 시 Gram Matrix 를 사용하는 이유로 옳은 것을 모두 고르시오.

1. 특징맵 간 상관관계를 반영하기 위해
2. 스타일의 질감/패턴 정보를 표현하기 위해
3. 이미지의 전체 구조를 보존하기 위해
4. 픽셀 단위의 직접적인 차이를 계산하기 위해

>> 1, 2

문제 2) 다음은 Neural Style Transfer(NST)의 전체 흐름을 나타낸 것이다.
올바른 수행 순서를 고르시오.

- ㄱ. 초기 생성 이미지(white noise 또는 content copy) 설정
- ㄴ. Content Loss / Style Loss 계산
- ㄷ. 콘텐츠 이미지와 스타일 이미지 입력
- ㄹ. VGG19 특징 추출
- ㅁ. 경사하강법을 통해 생성 이미지를 최적화

보기

1. ㄷ → ㄹ → ㄱ → ㄴ → ㅁ
2. ㄱ → ㄷ → ㄹ → ㄴ → ㅁ
3. ㄷ → ㄱ → ㄹ → ㄴ → ㅁ
4. ㄱ → ㄹ → ㄴ → ㄷ → ㅁ

>> 1